Embedded systems Architecture (BACSE324)

Assessment 9: Four-Way Traffic Light Control and LED Fading Using Arduino Uno

Lab Sl. No.	**9**	
Lab Slot	**L 45+ L46**	
Registration No.	**25BCE2640**	
Name:	**AARON ROBERTSON**	
Mobile no.:	**+91 9746244366**	
Title::	**Four-Way Traffic Light Control and LED Fading Using Arduino Uno**	
TRA Name:	**Mahesh Kumar**	Assigned date of Task:
TRA Mobile number:	**9952995367**	Date of Completion:
Faculty Name:	**DEBASHISH DASH**	Assessment no: 9
Email ID of student : aaron.robertson2025@vitstudent.ac.in		
VTOP uploading Date and time:		

Contents

Component	Page No.	Remarks
Program 1, 2	3, 8	
AIM of Experiment	3, 8	
Tools Required	3, 8	
Basic Brief Theory	3, 8	
Circuit Connections	3, 8	
Arduino Code with Comments	4, 9	
Calculations	5, 9	
Procedure to perform the AIM	5, 9	
Output Snips with name	6, 10	
Results	7, 11	
Inference	7, 11	

Signature of TRA

Program 1

Design a hardware circuit for a four-way traffic-light junction. Provide red, yellow and green LEDs for each direction and use 220 ohm resistors.

Aim of the Experiment

To interface twelve LEDs with Arduino Uno and operate a timed four-way traffic-light sequence that gives one approach a green signal while the others remain red.

Tools Required

- Arduino Uno R3
- Four red LEDs, four yellow LEDs and four green LEDs
- Twelve 220 ohm resistors, one for each LED
- Breadboard
- Jumper wires
- USB cable
- Computer with Arduino IDE

Brief Theory

A traffic controller assigns right of way through red, yellow and green signals. Red means stop, green permits movement and yellow signals the end of the current phase. This model serves North, East, South and West one at a time.
Twelve digital outputs independently drive twelve LEDs through individual 220 ohm series resistors. The proposed cycle uses 5 seconds of green, 2 seconds of yellow and 1 second with all directions red before the next approach is released. These timings are selected for the demonstration, not specified by the question.

Circuit Connections

Component terminal	Connection
North red LED anode	Arduino D2 through its own 220 ohm resistor
North yellow LED anode	Arduino D3 through its own 220 ohm resistor
North green LED anode	Arduino D4 through its own 220 ohm resistor
East red LED anode	Arduino D5 through its own 220 ohm resistor
East yellow LED anode	Arduino D6 through its own 220 ohm resistor
East green LED anode	Arduino D7 through its own 220 ohm resistor

South red LED anode	Arduino D8 through its own 220 ohm resistor
South yellow LED anode	Arduino D9 through its own 220 ohm resistor
South green LED anode	Arduino D10 through its own 220 ohm resistor
West red LED anode	Arduino D11 through its own 220 ohm resistor
West yellow LED anode	Arduino D12 through its own 220 ohm resistor
West green LED anode	Arduino D13 through its own 220 ohm resistor
All LED cathodes	Common ground rail connected to Arduino GND
Arduino USB port	Computer USB port

Wire each approach in red-yellow-green order. Use only one LED per output pin, check every LED polarity and switch off power before changing connections.

Arduino Code with Comments

```
// North, East, South, West pin order.
const byte redPins[4]    = {2, 5, 8, 11};
const byte yellowPins[4] = {3, 6, 9, 12};
const byte greenPins[4]  = {4, 7, 10, 13};
const unsigned long GREEN_TIME = 5000;
const unsigned long YELLOW_TIME = 2000;
const unsigned long ALL_RED_TIME = 1000;
void allRed() {
  for (byte i = 0; i < 4; i++) {
    digitalWrite(greenPins[i], LOW);
    digitalWrite(yellowPins[i], LOW);
  }
  for (byte i = 0; i < 4; i++) {
    digitalWrite(redPins[i], HIGH);
  }
}
void setup() {
  for (byte i = 0; i < 4; i++) {
    pinMode(redPins[i], OUTPUT);
    pinMode(yellowPins[i], OUTPUT);
    pinMode(greenPins[i], OUTPUT);
  }
  allRed();
  delay(ALL_RED_TIME);
}
void loop() {
  for (byte direction = 0; direction < 4; direction++) {
    digitalWrite(redPins[direction], LOW);
    digitalWrite(greenPins[direction], HIGH);
    delay(GREEN_TIME);
    digitalWrite(greenPins[direction], LOW);
    digitalWrite(yellowPins[direction], HIGH);
    delay(YELLOW_TIME);
    allRed();
    delay(ALL_RED_TIME);
  }
}
```

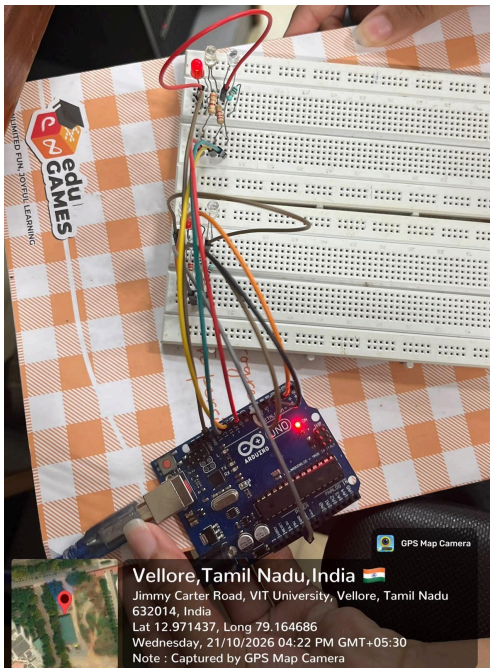
Manual calculation:

None

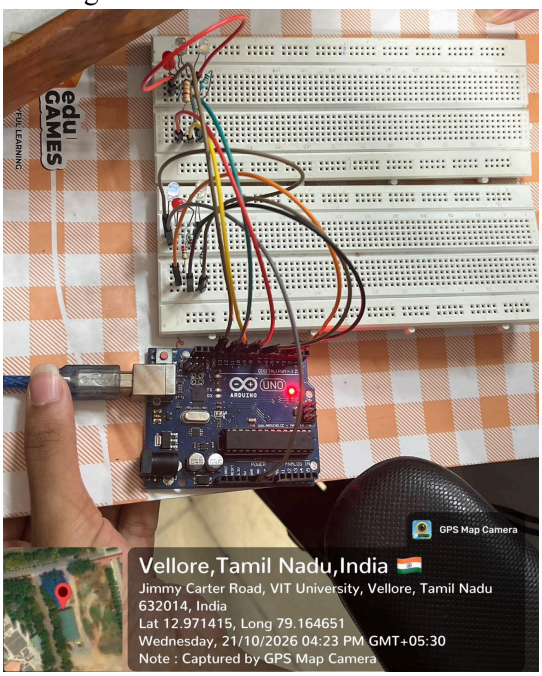
Procedure

1. Label the four approaches North, East, South and West and arrange three LEDs at each approach.
2. Disconnect power and wire each LED anode through a separate 220 ohm resistor to the pin listed in the connection table. Connect every cathode to GND.
3. Connect the Uno to the computer, select Arduino Uno and the correct port, and prepare the traffic-control sketch in the blank code section.
4. Configure D2 to D13 as outputs. Begin with all red LEDs ON and all yellow and green LEDs OFF.
5. For North, turn its red OFF and green ON for 5 seconds; keep East, South and West red.
6. Turn North green OFF and yellow ON for 2 seconds, then yellow OFF and red ON. Hold all four red signals for 1 second.
7. Repeat the same sequence for East, South and West, then repeat the complete cycle.
8. Verify and upload the completed sketch. Check that two approaches never show green together and record the actual observations.

Output Snips



Traffic-light circuit with the red LED illuminated.



Traffic-light circuit showing a different LED state.

Result

Expected result: North, East, South and West should receive green signals in turn. Each green phase should be followed by yellow and an all-red interval. All other approaches should remain red while one approach is active. Confirm the sequence on the physical circuit.

Inference

Arduino digital outputs can coordinate a multi-direction traffic-light sequence. Individual current-limiting resistors protect the LEDs, while a shared ground completes each circuit. The all-red interval separates successive phases and helps demonstrate how signal interlocking avoids conflicting green indications.

Program 2

Design an Arduino circuit that fades an LED using a PWM pin. Vary brightness from 0 to 255 and back to 0 using brightness and fadeAmount, and observe the effect of changing the delay.

Aim of the Experiment

To control LED brightness through Arduino PWM and produce a repeating fade-in and fade-out cycle by changing the duty cycle.

Tools Required

- Arduino Uno R3
- One LED
- One 220 ohm resistor
- Breadboard
- Jumper wires
- USB cable
- Computer with Arduino IDE

Brief Theory

Pulse Width Modulation switches a digital output rapidly between LOW and HIGH. Changing the fraction of time spent HIGH changes the average light output. On the Uno, analogWrite() accepts a value from 0 to 255: 0 keeps the LED off and 255 keeps it fully on.

Use PWM pin D9 and initialise brightness to 0 and fadeAmount to 5. After writing the brightness, change it by fadeAmount. Reverse the sign of fadeAmount at 0 or 255. A short delay between updates controls the apparent fading speed; a longer delay makes the fade slower.

Circuit Connections

Component terminal	Connection
Arduino D9 (PWM)	220 ohm resistor, then LED anode (long lead)
LED cathode (short lead)	Arduino GND through the common ground rail
Arduino USB port	Computer USB port

Use D9 because it supports PWM. A series resistor is required; do not connect the LED directly between the output pin and GND.

Arduino Code with Comments

```
// Fade the LED using PWM on pin 9.
const byte ledPin = 9;
int brightness = 0;
int fadeAmount = 5;
const unsigned long FADE_DELAY = 30;
void setup() {
  pinMode(ledPin, OUTPUT);
  analogWrite(ledPin, 0);
}
void loop() {
  analogWrite(ledPin, brightness);
  delay(FADE_DELAY);
  brightness = constrain(brightness + fadeAmount, 0, 255);
  if (brightness == 0 || brightness == 255) {
    fadeAmount = -fadeAmount;
  }
}
```

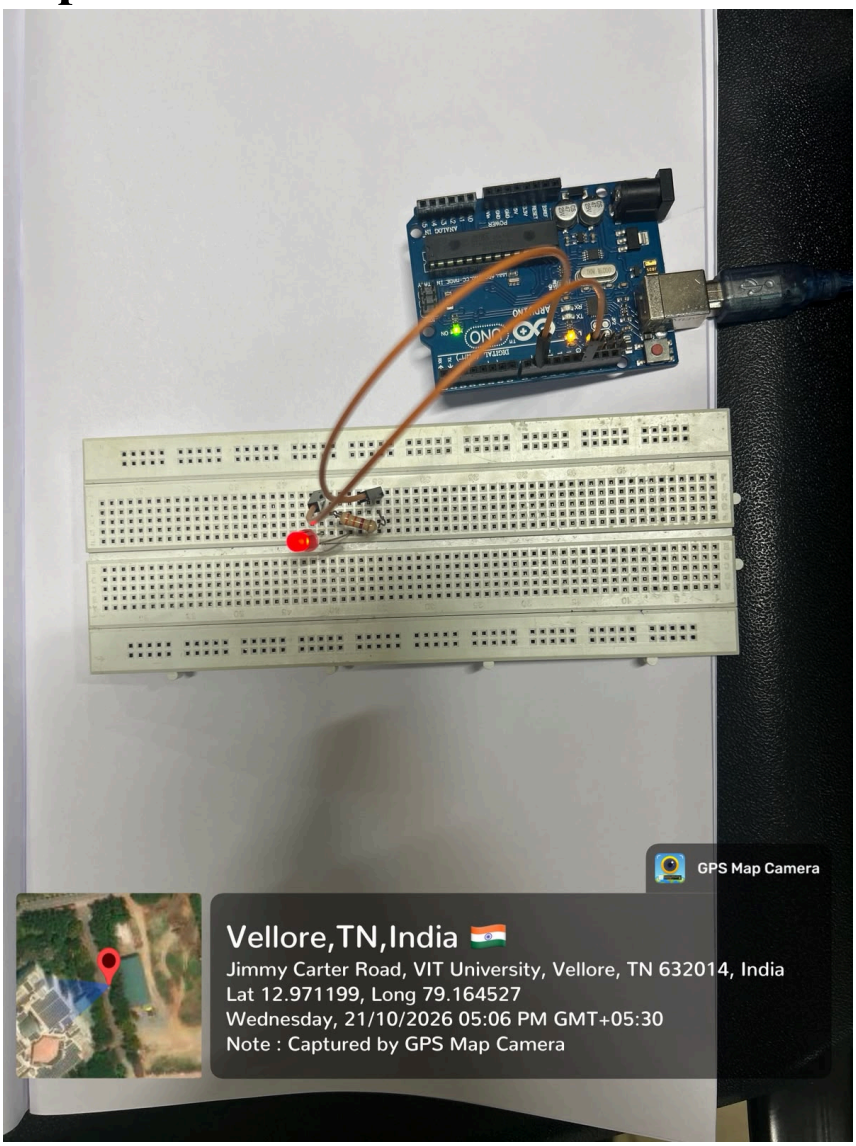
Manual calculation:

None

Procedure

1. Disconnect power. Connect D9 through the 220 ohm resistor to the LED anode and connect the cathode to Arduino GND.
2. Connect the board by USB, select Arduino Uno and the correct port, and prepare the fading sketch in the blank code section.
3. Declare brightness with an initial value of 0 and fadeAmount with a value of 5. Use D9 as the LED output.
4. Write brightness to D9 with analogWrite(), then add fadeAmount to brightness on each loop.
5. When brightness reaches 0 or 255, reverse the sign of fadeAmount so the next updates move in the opposite direction. Keep the value within 0 to 255.
6. Begin with a delay of 30 milliseconds between brightness updates. Verify and upload the completed sketch.
7. Observe the LED gradually becoming brighter and dimmer through several cycles.
8. Repeat with shorter and longer delays, such as 10 and 60 milliseconds, and compare the fading speeds. Record actual observations.

Output Snips



Single-LED circuit used for the PWM fading experiment.

Result

Expected result: The LED should gradually brighten from OFF to full brightness, then dim back to OFF repeatedly. Reversing fadeAmount at the limits should reverse the direction of fading. Increasing the update delay should slow the cycle; reducing it should speed up the effect.

Inference

PWM controls perceived LED brightness by varying duty cycle rather than generating a continuously variable output voltage. The brightness variable determines the PWM level, while fadeAmount determines the step direction and size. The delay between updates controls how quickly the brightness changes.

ESA Lab Assessment report

Sl.no.	Components	Marks Obtained
1	Pre Lab Work (Flow chart and Algorithm) documentation (2)	
2	Hardware or software Implementation (2)	
3	Manual calculation , Output data table / graph verification (2)	
4	Result with Inference (2)	
5	Post Lab Work (C Code with comments) documentation (2)	
	Total (10)	

1	Name: Aaron Robertson	
2	Reg. No.: 25BCE2640	
3	Lab slot: L 45+ L46	Lab. Slot Time: 3:50 PM – 5:30 PM
4	Assessment no.: 9	
5	Title of Assessment: Four-Way Traffic Light Control and LED Fading Using Arduino Uno	
6	Date of Assessment assigned	
7	Completion date	
8	VTOP Upload deadline	
9	Mobile number: 91+ 9746244366	
10	Email id:aaron.robertson2025@vitstudent.ac.in	
11	TRA Signature with Date	
12	Faculty Signature with Date	